Airflow Hacks to Trigger FC & Vortex — and Optimize the Workflow

White paper for `airflow-fc-vortex`

Subtitle: A practical control plane for Fusion Compiler → Vortex forensics, designed to be understandable in one sitting (unlike kernel AutoFDO).

Companion: `HACKS.md`, `scripts/local_runner.py`, `dags/`, `results/SUMMARY.txt`

Abstract

Physical-design teams already know how to launch Fusion Compiler and how to grep logs. What they lack is a **shared workflow** that (a) always runs Vortex-style policy checks after FC, (b) routes on severity, (c) respects license seats, and (d) lets methodology add corners without editing tribal Bash.

This paper presents ten Airflow "hacks" — really: standard Airflow features applied deliberately — with a **laptop runner that needs no Airflow install**. Mocks stand in for FC and Vortex; production swaps env vars to real binaries. A measured demo shows naive JSON greps **false-positive on clean logs**, while branching on Vortex `overall` does not.

---

1. Motivation (keep it simple)

Old world	New world
Cron + Bash	DAG with params
Grep folklore	Policy YAML + `overall`
Hope licenses exist	Airflow **Pool**
"Who ran corner X?"	Mapped tasks / JSON jobs

If AutoFDO was "optimize the CPU under the job," this project is "**optimize the assembly line that runs the job**."

---

2. Architecture

Mocks: `mock_tools/fc_mock.py`, `mock_tools/vortex_mock.py`.
Real: `FC_BIN`, `VORTEX_BIN`, `VORTEX_POLICY`.

3. The ten hacks (detail)

See also HACKS.md.

1. **Chain** — pass log path between tasks (XCom / return dict).
2. **Branch** — `@task.branch` on `overall ∈ {fatal,error,warning,clean}`.
3. **Pool** — `pool="fc_licenses"`; slots = purchased seats.
4. **Dynamic mapping** — `fc_and_vortex.expand(job=corners)`.
5. **Short-circuit** — skip Vortex when FC returns hard-fatal.
6. **Dataset** — schedule Vortex DAG on log landing.
7. **Retries** — license checkout blips (`retries`, `retry_delay`).
8. **Factory** — `config/workflows.json` lists jobs.
9. **SLA** — attach SLA + callback on FC task (pattern in paper; wire to your notifier).
10. **Params** — UI/API trigger with design/corner.

---

4. Experimental design

Piece	Role
`local_runner.py`	Executes hacks 1–5,7–8 without Airflow
`adhoc/bash_fc_vortex_pipeline.sh`	Ad-hoc baseline; `NAIVE=1` shows grep trap
`examples/compare_all.sh`	LOC + outcomes + CLAIM_GATE
`dags/*.py`	Drop-in Airflow 2.7+ DAGs

---

5. Measured results

Regenerate: `./examples/compare_all.sh`.

Typical outcomes on the authoring host:

- Chain branches to `open_jira_timing` on timing_fail.
- Fatal FC → `skipped_vortex=True`.
- License flake → `retries≥1` then success.
- Pool slots=1 serializes multi-corner wall time vs unlimited parallel.
- **Naive grep false positives** on clean corners (rule names present in JSON).
- Fair `overall` routing archives clean corners.

CLAIM_GATE: `CITEABLE=yes_airflow_hacks_demo`.

---

6. Dimension table

Dimension	Airflow + Vortex policy	Ad-hoc Bash/cron
Trigger FC	Parametrized DAG	Script argv / tribal
Trigger Vortex	Always-on task	“Remember to run it”
Severity route	Branch on `overall`	Grep (fragile)
Licenses	Pool	Hope / manual
Add corner	JSON / params	Edit script
Who can review?	Methodology + CAD	Bash experts
Audit	Airflow run history	Scrollback

7. Monday morning playbook

1. `pip install pyyaml && ./examples/compare_all.sh` — trust the demo.
2. Stand up Airflow (or use existing).
3. Create pool `fc_licenses` = seat count.
4. Copy `dags/` → `$AIRFLOW_HOME/dags`.
5. Point `VORTEX_POLICY` at your real YAML; optionally `FC_BIN` / `VORTEX_BIN`.
6. Trigger `fc_vortex_chain` with a real design/corner.
7. Wire `page_oncall` / `open_jira_timing` EmptyOperators to real notifiers.

---

8. Limitations

- Mocks ≠ Synopsys QoR or Vortex GB perf.
- Demo wall times are illustrative.
- Dataset DAG needs Airflow 2.4+ and a producer that updates the Dataset.
- SLA callbacks need your email/Slack/PagerDuty hookup.

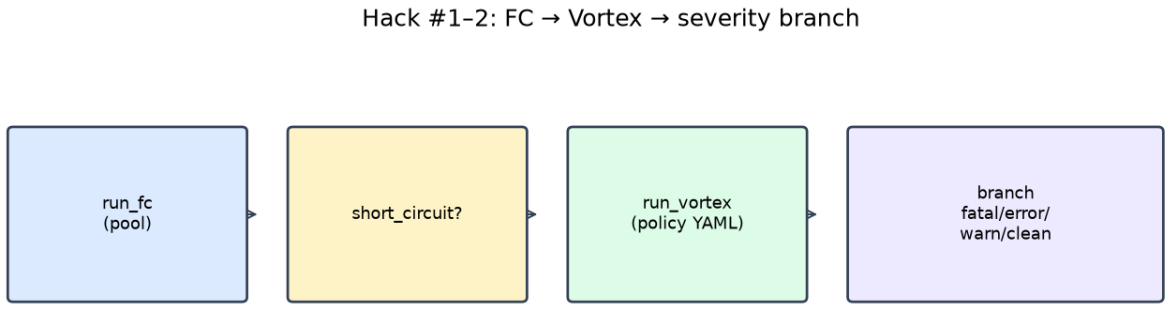
---

9. Conclusion

Optimizing PD workflows does not start with kernel compilers. It starts with a **control plane**: FC triggers Vortex, severity decides the next human action, licenses are pooled, corners are data. This repository makes that plane **copy-pasteable** and **demoable on a laptop**.

Appendix — glossary

Term	Meaning
Pool	Airflow concurrency limit (use for licenses)
XCom	Small metadata pass between tasks (log path)
overall	Vortex rollup severity
CLAIM_GATE	Honesty label for measured demos
local_runner	Airflow-free executor of the same hacks



Ten Airflow hacks (CAD / methodology)

1 Chain FC→Vortex	2 Severity branch
3 License pool	4 Map corners
5 Short-circuit	6 Dataset on log
7 Retry/backoff	8 Workflow JSON
9 SLA alert	10 UI params